LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3

SINGLE LINK LIST

Oleh:

Muhammad Rafly Ash-Shadiq NIM. 2510817310007

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Link List ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rafly Ash-Shadiq
NIM : 2510817310007

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Pembahasan 5
SOAL 2 21
A. Pembahasan 21
SOAL 3 23
A. Pembahasan 23
SOAL 4 25
A. Pembahasan 25
SOAL 5 27
A. Pembahasan 27
SOAL 6 29
A. Pembahasan 29
SOAL 7 31
A. Pembahasan 31
SOAL 8 33
A. Pembahasan 33
SOAL 9 35
A. Pembahasan 35
SOAL 10 37
A. Pembahasan 37
LINK GITHUB 38

DAFTAR GAMBAR

Gambar 1 Screenshot Output Soal 1 16
Gambar 2 Screenshot Output Soal 2 21
Gambar 3 Screenshot Output Soal 3 23
Gambar 4 Screenshot Output Soal 4 25
Gambar 5 Screenshot Output Soal 5 27
Gambar 6 Screenshot Output Soal 6 29
Gambar 7 Screenshot Output Soal 7 31
Gambar 8 Screenshot Output Soal 8 33

DAFTAR TABEL

Table 1 Source Code Soal Nomor 1 5

SOAL 1

1. Cobalah program berikut, running, simpan program, dan screenshoot hasil running!

```cpp
#include <conio.h>
#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct TNode {
    string data;
    TNode *next;
};

TNode *head, *tail;

int pil;
char pilihan[2];
string dataBaru, data;

void init();
int isEmpty();

void tambahDepan();
void tambahBelakang();
void hapusDepan();
void hapusBelakang();
void tampilkan();
void reset();
void cariData();
void hapusData();
void sisipkanSebelum();
void sisipkanSetelah();

int main()
{
    do {
        cout<<"Single Linked List Circular (SLLC)"<<endl;
        cout<<"================================"<<endl;
        cout<<"1. Tambah Depan"<<endl;
        cout<<"2. Tambah Belakang"<<endl;
        cout<<"3. Hapus Depan"<<endl;
        cout<<"4. Hapus Belakang"<<endl;
        cout<<"5. Tampilkan Data"<<endl;
        cout<<"6. Hapus Semua Elemen"<<endl;
        cout<<"7. Cari Data"<<endl;
        cout<<"8. Hapus Setiap Data Tertentu"<<endl;
        cout<<"9. Sisipkan Node/Data Baru Sebelum Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
```

```
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<data<<"\" yang berada di depan telah berhasil dihapus."<<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<data<<"\" yang berada di belakang telah berhasil dihapus."<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
        case 7:
            cariData();
            break;
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Nama (NIM)."<<endl;
        }

        cout<<"\nPress any key to continue!"<<endl;
        getch();
        system("cls");

    } while (pil<11);
```

```
}

    void init(){
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
            head = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
    }

    void tambahBelakang() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            tail->next = baru;
            tail = baru;
```

```
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian belakang."<<endl;
    }

    void hapusDepan() {
        if(isEmpty() == 0) {
            TNode *hapus;
            hapus = head;
            data = hapus->data;

            if(head != tail) {
                head = head->next;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void hapusBelakang() {
        if(isEmpty() == 0) {
            TNode *hapus, *newTail;
            hapus = tail;
            data = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
```

```
            bantu = head;

            do {
                cout<<bantu->data<<' ';
                bantu = bantu->next;
            } while(bantu != head);
            cout<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void reset() {
        if(isEmpty() == 0) {
            TNode *bantu, *hapus;
            bantu = head;

            do {
                hapus = bantu;
                bantu = bantu->next;
                delete hapus;
            } while(bantu != head);

            init();
            cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void cariData() {
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukkan data yang ingin dicari : ";
            cin>>cari;

            TNode *bantu, *hapus, *newTail, *bantuTampilkan;
            bool apaDitemukan = false;

            bantu = head;

            do {
                if(cari == bantu->data){
                    cout<<"Setiap data yang berada di dalam tanda kurung siku ([...]) "
                        <<"merupakan data yang anda cari."<<endl;
                    cout<<"Linked List : ";
                    bantuTampilkan = head;

                    do {
                        if(cari == bantuTampilkan->data)
```

```
                    cout<<"["<<bantuTampilkan->data<<"] ";
                else
                    cout<<bantuTampilkan->data<<' ';
                bantuTampilkan = bantuTampilkan->next;
            } while(bantuTampilkan != head);

            apaDitemukan = true;
            cout<<endl;
            break;
        }
        bantu = bantu->next;
    } while(bantu != head);

    if(apaDitemukan == false)
        cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;
} else cout<<"Tidak terdapat data pada Linked List."<<endl;
}

void hapusData() {
    if(isEmpty() == 0) {
        string cari;
        cout<<"Masukkan data yang ingin dihapus : ";
        cin>>cari;

        TNode *bantu, *sebelum, *hapus[255], *bantu2;
        int hitung = 0;
        bool apaDitemukan = false;

        bantu = head;

        do {
            bantu2 = bantu;
            if(cari == bantu->data){
                hapus[hitung++] = bantu;
                apaDitemukan = true;
                if(bantu != head && bantu != tail) {
                    sebelum->next = bantu->next;
                    bantu2 = sebelum;
                }
            }
            sebelum = bantu2;
            bantu = bantu->next;
        } while(bantu != head);

        if(apaDitemukan == true) {
            for(int i = 0; i < hitung; i++) {
```

```
                if(hapus[i] == head){
                    hapusDepan();
                } else if(hapus[i] == tail) {
                    hapusBelakang();
                } else {
                    delete hapus[i];
                }
            }

            cout<<"Setiap data \""<<cari<<"\" yang terdapat pada Linked List telah dihapus";
        } else cout<<"Data \""<<cari<<"\" tidak ditemukan pada Linked List."<<endl;

    } else cout<<"Tidak terdapat data pada Linked List."<<endl;
}

void sisipkanSebelum() {
    if(isEmpty() == 0) {
        TNode *bantu, *sebelum;
        string nextData;
        bool apaAda;

        bantu = head;
        sebelum = tail;

        cout<<"Sisipkan data baru sebelum data : ";
        cin>>nextData;

        do {
            if(nextData == bantu->data){
                apaAda = true;
                break;
            } else {
                sebelum = bantu;
                bantu = bantu->next;
            }
        } while(bantu != head);

        if(apaAda==true) {
            cout<<"Masukkan data yang ingin ditambahkan : ";
            cin>>dataBaru;

            TNode *baru;
            baru = new TNode;

            baru->data = dataBaru;
```

```
                baru->next = bantu;

                sebelum->next = baru;

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru setelah data : ";
            cin>>prevData;

            do {
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;

```

```
                bantu->next = baru;

                if(bantu == tail){
                    tail = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setalah data \""<<prevData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

```

A. Pembahasan

Table 1 Source Code Soal Nomor 1

```
1.  #include <conio.h>
2.  #include <iostream>
3.  #include <stdlib.h>
4.
5.  using namespace std;
```

```
6.
7.  struct TNode{
8.      string data;
9.      TNode *next;
10. };
11.
12. TNode *head, *tail;
13.
14. int pil;
15. char pilihan[2];
16. string dataBaru, dataHapus;
17.
18. void init();
19. int isEmpty();
20.
21. void tambahDepan();
22. void tambahBelakang();
23. void hapusDepan();
24. void hapusBelakang();
25. void tampilkan();
26. void reset();
27. void cariData();
28. void hapusData();
29. void sisipkanSebelum();
30. void sisipkanSetelah();
31.
32. int main()
33. {
34.     do {
35.         cout<<"Single Linked
    List Circular (SLLC)"<<endl;
36.         cout<<"==================================="
    <<endl;
37.         cout<<"1. Tambah Depan"<<endl;
38.         cout<<"2. Tambah Belakang"<<endl;
39.         cout<<"3. Hapus Depan"<<endl;
40.         cout<<"4. Hapus Belakang"<<endl;
41.         cout<<"5. Tampilkan Data"<<endl;
42.         cout<<"6. Hapus Semua Elemen"<<endl;
43.         cout<<"7. Cari Data"<<endl;
44.         cout<<"8. Hapus Setiap Data Tertentu
    "<<endl;
45.         cout<<"9. Sisipkan Node/Data Baru
```

```
Sebelum Data Tertentu"<<endl;
        cout<<"10. Sisipkan Node/Data Baru Setelah Data Tertentu"<<endl;
        cout<<"11. Quit"<<endl;
        cout<<"Pilihan : ";
        cin>>pilihan;
        pil=atoi(pilihan);

        switch(pil) {
        case 1:
            tambahDepan();
            break;
        case 2:
            tambahBelakang();
            break;
        case 3:
            hapusDepan();
            cout<<"Data \""<<dataHapus<<"\" yang berada di depan telah berhasil dihapus." <<endl;
            break;
        case 4:
            hapusBelakang();
            cout<<"Data \""<<dataHapus<<"\" yang berada di belakang telah berhasil dihapus. "<<endl;
            break;
        case 5:
            tampilkan();
            break;
        case 6:
            reset();
            break;
        case 7:
            cariData();
            break;
        case 8:
            hapusData();
            break;
        case 9:
            sisipkanSebelum();
            break;
        case 10:
```

```
            sisipkanSetelah();
            break;
        default:
            cout<<"\nTERIMA KASIH"<<endl;
            cout<<"Program was made by Muhammad Rafly Ash-Shadiq (2510817310007)."<<endl;
        }

        cout<<"\nPress any key to continue!"<<endl;
        getch();
        system("cls");

    } while (pil < 11);
}

    void init() {
        head = NULL;
        tail = NULL;
    }

    int isEmpty() {
        if(head == NULL) return 1;
        else return 0;
    }

    void tambahDepan() {
        cout<<"Masukkan data : ";
        TNode *baru;
        baru = new TNode;
        cin>>dataBaru;
        baru->data = dataBaru;
        baru->next = baru;

        if(isEmpty() == 1) {
            head = baru;
            tail = baru;
        } else {
            baru->next = head;
            head = baru;
            tail->next = head;
        }
        cout << "Data \""<<dataBaru<<"\" berhasil dimasukkan di bagian depan."<<endl;
```

```
124.     }
125.
126.     void tambahBelakang() {
127.         cout<<"Masukkan data : ";
128.         TNode *baru;
129.         baru = new TNode;
130.         cin>>dataBaru;
131.         baru->data =  dataBaru;
132.         baru->next = baru;
133.
134.         if(isEmpty() == 1) {
135.             head = baru;
136.             tail = baru;
137.         } else {
138.             tail->next = baru;
139.             tail = baru;
140.             tail->next = head;
141.         }
142.         cout << "Data \""<<dataBaru<<"\"
     berhasil dimasukkan dibagian belakang."<<endl;
143.     }
144.
145.     void hapusDepan() {
146.         if(isEmpty() == 0) {
147.             TNode *hapus;
148.             hapus = head;
149.             dataHapus = hapus->data;
150.
151.             if(head != tail) {
152.                 head = head->next;
153.                 tail->next = head;
154.             } else {
155.                 init();
156.             }
157.
158.             delete hapus;
159.         } else cout<<"Tidak terdapat data
     pada Linked List."<<endl;
160.     }
161.
162.     void hapusBelakang(){
163.         if(isEmpty() == 0) {
164.             TNode *hapus, *newTail;
```

```
            hapus = tail;
            dataHapus = hapus->data;

            if(head != tail) {
                newTail = head;
                while(newTail->next != tail) {
                    newTail = newTail->next;
                }
                tail = newTail;
                tail->next = head;
            } else {
                init();
            }

            delete hapus;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void tampilkan() {
        if(isEmpty() == 0) {
            TNode *bantu;
            bantu = head;

            do {
                cout<<bantu->data<<' ';
                bantu = bantu->next;
            } while(bantu != head);
            cout<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void reset() {
        if(isEmpty() == 0) {
            TNode *bantu, *hapus;
            bantu = head;

            do {
                hapus = bantu;
                bantu = bantu->next;
                delete hapus;
            } while(bantu != head);
```

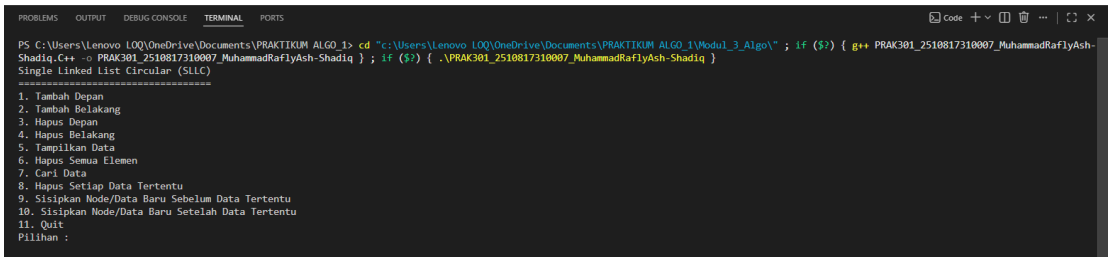
```

            init();
            cout<<"Seluruh elemen pada Linked List telah dibersihkan."<<endl;
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }

    void cariData(){
        if(isEmpty() == 0) {
            string cari;
            cout<<"Masukan data yang ingin dicari : ";
            cin>>cari;

            TNode *bantu, *hapus, *newTail, *bantuTampilkan;
            bool apaDitemukan = false;

            bantu = head;

            do {
                if(cari == bantu->data){
                    cout<<"Setiap data yang berada di dalam tanda kurung siku ([...])"
                        <<"merupakan data yang anda cari."<<endl;
                    cout<<"Linked List : ";
                    bantuTampilkan = head;

                    do {
                        if(cari == bantuTampilkan->data)
                            cout<<"["<<bantuTampilkan->data<<"] ";
                        else
                            cout<<bantuTampilkan->data<<' ';
                        bantuTampilkan = bantuTampilkan->next;
                    } while(bantuTampilkan != head);
```

```
237.
238.                    apaDitemukan = true;
239.                    cout<<endl;
240.                    break;
241.                }
242.                bantu = bantu->next;
243.            } while(bantu != head);
244.
245.            if(apaDitemukan == false)
246.                cout<<"Data \""
    <<cari<<"\" tidak ditemukan pada
    Linked List."<<endl;
247.        } else cout<<"Tidak terdapat
    data pada Linked List."<<endl;
248.    }
249.
250.    void hapusData(){
251.        if(isEmpty() == 0) {
252.            string cari;
253.            cout<<"Masukkan data
    yang ingin dihapus : ";
254.            cin>>cari;
255.
256.            TNode *bantu,
    *sebelum, *hapus[225], *bantu2;
257.            int hitung = 0;
258.            bool apaDitemukan = false;
259.
260.            bantu = head;
261.
262.            do {
263.                bantu2 = bantu;
264.                if(cari == bantu->data){
265.                    hapus[hitung++] = bantu;
266.                    apaDitemukan = true;
267.                    if(bantu != head &&
    bantu != tail) {
268.                        sebelum->next =
     bantu->next;
269.                        bantu2 = sebelum;
270.                    }
271.                }
272.                sebelum = bantu2;
```

```
273.                 bantu = bantu->next;
274.             } while(bantu != head);
275.
276.             if(apaDitemukan == true) {
277.                 for(int i = 0; i < hitung; i++) {
278.                     if(hapus[i] == head){
279.                         hapusDepan();
280.                     } else if(hapus[i] == tail) {
281.                         hapusBelakang();
282.                     } else {
283.                         delete hapus[i];
284.                     }
285.                 }
286.
287.                 cout<<"Setiap data \""
    <<cari<<"\" yang terdapat pada
    Linked List telah dihapus";
288.             } else cout<<"Data \""
    <<cari<<"\" tidak ditemukan pada
    Linked List."<<endl;
289.
290.         } else cout<<"Tidak terdapat
    data pada Linked List."<<endl;
291.     }
292.
293.     void sisipkanSebelum() {
294.         if(isEmpty() == 0) {
295.             TNode *bantu, *sebelum;
296.             string nextData;
297.             bool apaAda;
298.
299.             bantu = head;
300.             sebelum = tail;
301.
302.             cout<<"Sisipkan data
    baru sebelum data : ";
303.             cin>>nextData;
304.
305.             do {
306.                 if(nextData == bantu->data){
307.                     apaAda = true;
308.                     break;
309.                 } else {
```

```
                    sebelum = bantu;
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu;

                sebelum->next = baru;

                if(bantu == head){
                    head = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan sebelum data \""<<nextData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<nextData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak tedapat data pada Linked List."<<endl;
    }

    void sisipkanSetelah() {;
        if(isEmpty() == 0) {
            TNode *bantu;
            string prevData;
            bool apaAda;

            bantu = head;

            cout<<"Sisipkan data baru
```

```
setelah data : ";
            cin>>prevData;

            do {
                if(prevData == bantu->data){
                    apaAda = true;
                    break;
                } else {
                    bantu = bantu->next;
                }
            } while(bantu != head);

            if(apaAda==true) {
                cout<<"Masukkan data yang ingin ditambahkan : ";
                cin>>dataBaru;

                TNode *baru;
                baru = new TNode;

                baru->data = dataBaru;
                baru->next = bantu->next;

                bantu->next = baru;

                if(bantu == tail){
                    tail = baru;
                }

                cout << "Data \""<<dataBaru<<"\" berhasil disisipkan setelah data \""<<prevData<<"\"."<<endl;
            } else {
                cout<<"Tidak terdapat data \""<<prevData<<"\" pada Linked List."<<endl;
            }
        } else cout<<"Tidak terdapat data pada Linked List."<<endl;
    }
```



Gambar 1 Screenshot Output Soal 1

Pada baris 1 #include <conio.h> digunakan untuk fungsi seperti getch() yang berfungsi menahan layar sebelum lanjut. Pada baris 2 #include <iostream> digunakan untuk proses input dan output seperti cin dan cout. Pada baris 3 #include <stdlib.h> digunakan untuk fungsi atoi() yaitu mengubah input karakter menjadi integer. Pada baris 5 using namespace std; digunakan agar tidak perlu menulis std:: berulang kali. Pada baris 7–10 dideklarasikan struct TNode yang merupakan node dalam linked list, berisi data bertipe string dan pointer next yang menunjuk ke node berikutnya. Pada baris 12 dideklarasikan pointer head dan tail sebagai penunjuk awal dan akhir linked list. Pada baris 14–16 dideklarasikan variabel global seperti pil (menu), pilihan (input menu dalam bentuk char), serta dataBaru dan dataHapus untuk menyimpan data sementara. Pada baris 18–30 merupakan deklarasi fungsi-fungsi yang akan digunakan, seperti tambah, hapus, tampil, dan lainnya. Pada baris 32–95 merupakan fungsi utama main(). Program menggunakan perulangan do-while untuk menampilkan menu secara terus menerus sampai pengguna memilih keluar. Pada baris 49 input menu dimasukkan ke variabel pilihan, lalu pada baris 50 diubah menjadi integer menggunakan atoi() agar bisa diproses di switch. Pada baris 52–88 digunakan switch-case untuk menentukan aksi sesuai pilihan user, seperti menambah, menghapus, menampilkan, dan lain-lain. Pada baris 90–92 digunakan getch() untuk pause dan system("cls") untuk membersihkan layar. Pada baris 97–100 fungsi init() digunakan untuk mengosongkan linked list dengan menjadikan head dan tail bernilai NULL. Pada baris 102–105 fungsi isEmpty() digunakan untuk mengecek apakah linked list kosong (head == NULL). Pada baris 107–124 fungsi tambahDepan() digunakan untuk menambah data di depan.

Node baru dibuat, lalu jika kosong menjadi head dan tail, jika tidak maka disambungkan ke depan dan tail diarahkan kembali ke head. Pada baris 126–143 fungsi tambahBelakang() digunakan untuk menambah data di belakang. Jika kosong, node jadi head dan tail, jika tidak maka ditambahkan setelah tail dan pointer diperbarui. Pada baris 145–160 fungsi hapusDepan() digunakan untuk menghapus node pertama. Jika lebih dari satu node, head dipindahkan ke berikutnya, jika hanya satu maka list dikosongkan. Pada baris 162–181 fungsi hapusBelakang() digunakan untuk menghapus node terakhir dengan mencari node sebelum tail, lalu memperbarui pointer. Pada baris 183–194 fungsi tampilkan() digunakan untuk menampilkan seluruh data dengan perulangan do-while sampai kembali ke head. Pada baris 196–210 fungsi reset() digunakan untuk menghapus seluruh node dengan cara menghapus satu per satu lalu mengosongkan list. Pada baris 212–248 fungsi cariData() digunakan untuk mencari data tertentu dan menandai data yang ditemukan dalam output. Pada baris 250–291 fungsi hapusData() digunakan untuk menghapus semua node dengan nilai tertentu dengan cara menelusuri dan menghapus node yang sesuai. Pada baris 293–336 fungsi sisipkanSebelum() digunakan untuk menambahkan node sebelum data tertentu dengan mengatur pointer antar node. Pada baris 338–379 fungsi sisipkanSetelah() digunakan untuk menambahkan node setelah data tertentu.

Alur program secara keseluruhan dimulai dari fungsi main(), di mana program akan menampilkan menu secara berulang menggunakan perulangan *do-while*. Pengguna diminta memasukkan pilihan dalam bentuk karakter yang kemudian dikonversi menjadi bilangan menggunakan fungsi atoi(). Hasil konversi ini digunakan dalam struktur switch-case untuk menentukan function mana yang akan dijalankan. Setelah satu proses selesai, program tidak langsung berhenti, melainkan kembali ke menu utama hingga pengguna memilih opsi keluar. Dengan demikian, seluruh function yang ada saling terhubung melalui kontrol utama di dalam main().

Sebelum masuk ke proses manipulasi data, program memiliki function init() dan isEmpty() sebagai dasar pengelolaan struktur data. Function init() berfungsi untuk menginisialisasi linked list dengan menjadikan head dan tail bernilai NULL, yang

berarti belum ada data sama sekali. Sementara itu, function isEmpty() digunakan untuk mengecek apakah linked list kosong atau tidak. Jika head bernilai NULL, maka function mengembalikan nilai 1 (kosong), dan jika tidak maka mengembalikan nilai 0 (tidak kosong). Function ini sangat penting karena hampir semua operasi seperti tambah, hapus, dan tampil selalu diawali dengan pengecekan kondisi ini agar program tidak mengalami kesalahan saat dijalankan.

Ketika pengguna memilih untuk menambah data di bagian depan, program akan menjalankan function tambahDepan(). Pada proses ini, program terlebih dahulu meminta input data, kemudian membuat node baru dan mengisi data tersebut ke dalam node. Node baru awalnya diarahkan ke dirinya sendiri untuk mengantisipasi kondisi circular saat data pertama dimasukkan. Jika linked list dalam keadaan kosong, maka node tersebut langsung menjadi head dan tail. Namun jika sudah terdapat data, maka node baru akan diarahkan ke head lama, kemudian posisi head diperbarui menjadi node baru, dan tail tetap diarahkan ke head agar struktur circular tetap terjaga. Dengan alur ini, setiap data baru selalu masuk di posisi paling depan.

Selanjutnya, jika pengguna memilih menambah data di bagian belakang, program akan menjalankan function tambahBelakang(). Proses awalnya hampir sama, yaitu membuat node baru dan mengisi data. Jika linked list masih kosong, node baru menjadi head dan tail. Namun jika sudah ada data, maka node baru akan ditambahkan setelah tail, kemudian posisi tail diperbarui menjadi node tersebut, dan tail kembali diarahkan ke head untuk menjaga hubungan melingkar. Dengan demikian, data baru selalu ditambahkan di bagian akhir tanpa merusak struktur yang sudah ada.

Pada saat pengguna memilih menghapus data di bagian depan, function hapusDepan() akan dijalankan. Program terlebih dahulu mengecek apakah linked list kosong. Jika kosong, maka akan ditampilkan pesan bahwa tidak ada data. Jika tidak kosong, maka node pertama (head) akan disimpan sementara untuk dihapus. Jika hanya terdapat satu node, maka linked list akan dikosongkan kembali dengan memanggil init(). Namun jika terdapat lebih dari satu node, maka posisi head dipindahkan ke node berikutnya, dan tail diarahkan kembali ke head baru. Setelah itu, node lama dihapus

dari memori. Proses ini memastikan bahwa penghapusan tidak merusak hubungan antar node.

Untuk penghapusan data di bagian belakang, function hapusBelakang() digunakan. Program akan mengecek apakah data kosong, lalu jika tidak kosong akan mencari node sebelum tail dengan melakukan penelusuran dari head. Node sebelum tail tersebut kemudian dijadikan tail baru, dan diarahkan kembali ke head. Jika hanya terdapat satu node, maka linked list akan dikosongkan. Setelah itu, node lama dihapus dari memori. Proses ini sedikit lebih kompleks karena harus mencari node sebelumnya terlebih dahulu.

Function tampilkan() digunakan untuk menampilkan seluruh data dalam linked list. Jika data tidak kosong, program akan mulai dari head dan menelusuri setiap node menggunakan perulangan *do-while*. Perulangan ini digunakan karena struktur bersifat circular, sehingga proses akan terus berjalan hingga kembali lagi ke head. Setiap data akan ditampilkan satu per satu sehingga pengguna dapat melihat isi keseluruhan linked list.

Selain itu, terdapat function reset() yang berfungsi untuk menghapus seluruh data dalam linked list. Program akan menelusuri setiap node dan menghapusnya satu per satu hingga kembali ke head, kemudian memanggil init() untuk mengosongkan struktur sepenuhnya. Function ini digunakan ketika pengguna ingin membersihkan semua data sekaligus.

Function cariData() digunakan untuk mencari data tertentu dalam linked list. Program akan meminta input data yang ingin dicari, kemudian menelusuri seluruh node mulai dari head. Jika data ditemukan, maka program akan menampilkan seluruh isi linked list dan memberikan tanda khusus pada data yang dicari, sehingga mudah dikenali. Jika data tidak ditemukan setelah satu putaran penuh, maka akan ditampilkan pesan bahwa data tidak ada. Proses ini menunjukkan mekanisme pencarian secara linear pada struktur circular.

Untuk penghapusan data tertentu, digunakan function hapusData(). Program akan meminta input data yang ingin dihapus, kemudian menelusuri seluruh node untuk

mencari data yang sesuai. Jika ditemukan, node tersebut akan dihapus dan pointer antar node diperbaiki agar tidak terputus. Proses ini dapat menghapus lebih dari satu data jika terdapat nilai yang sama. Jika data tidak ditemukan, maka program akan memberikan informasi kepada pengguna.

Terakhir, terdapat function sisipkanSebelum() dan sisipkanSetelah() yang digunakan untuk menambahkan data di posisi tertentu. Pada sisipkanSebelum(), program mencari node target, kemudian menyisipkan node baru sebelum node tersebut dengan mengatur pointer sebelumnya dan berikutnya. Jika node target adalah head, maka posisi head juga diperbarui. Sedangkan pada sisipkanSetelah(), node baru ditambahkan setelah node target, dan jika node target adalah tail, maka tail akan diperbarui. Kedua function ini menunjukkan bagaimana manipulasi pointer dilakukan untuk menyisipkan data di tengah struktur tanpa merusak hubungan yang ada.

Secara keseluruhan, semua function dalam program ini saling terhubung melalui penggunaan head, tail, dan pointer next. Setiap operasi selalu diawali dengan pengecekan kondisi menggunakan isEmpty(), kemudian dilakukan manipulasi data sesuai kebutuhan, dan diakhiri dengan penyesuaian pointer agar struktur circular tetap terjaga. Dengan alur seperti ini, program mampu mengelola data secara dinamis tanpa kehilangan keterhubungan antar elemen.

SOAL 2

Lakukan tambah data depan 3, 4, 7, 9, 10, 12 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Pembahasan

Gambar 2 Screenshot Output Soal 2

Pada soal nomor 2 ini, proses yang dilakukan adalah menambahkan data ke dalam struktur Single Linked List Circular (SLLC) menggunakan metode tambah depan, kemudian menampilkan seluruh data yang telah dimasukkan. Pengguna secara berurutan memilih menu tambah depan dan memasukkan nilai 3, 4, 7, 9, 10, dan 12. Setiap kali data baru dimasukkan dengan metode ini, posisi data tersebut akan menjadi node pertama (head), sehingga data sebelumnya akan bergeser ke belakang. Hal ini menyebabkan urutan data di dalam linked list menjadi kebalikan dari urutan saat input dilakukan.

Secara logika, ketika pertama kali dimasukkan nilai 3, maka linked list hanya berisi satu elemen yaitu 3. Ketika nilai 4 dimasukkan, maka 4 akan menjadi head baru dan menunjuk ke 3, sehingga urutannya menjadi 4 lalu 3. Proses ini terus berlanjut hingga data terakhir yaitu 12 dimasukkan, sehingga susunan akhir linked list menjadi

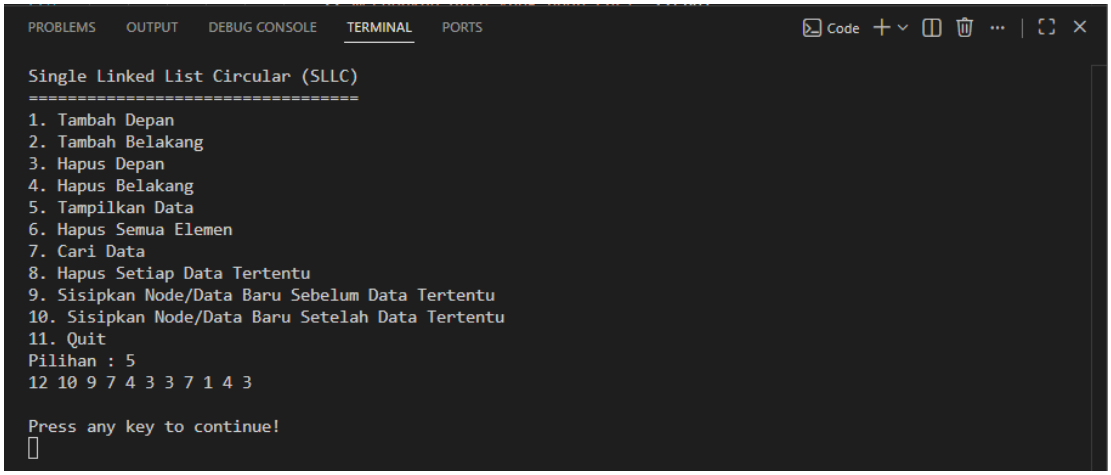
12, 10, 9, 7, 4, dan 3. Setelah itu, ketika pengguna memilih menu tampilkan data, program akan menelusuri seluruh node mulai dari head hingga kembali lagi ke head karena sifatnya yang melingkar.

Hasil output yang ditampilkan yaitu “12 10 9 7 4 3” menunjukkan bahwa proses penambahan data di bagian depan berjalan dengan benar. Selain itu, fungsi penelusuran data juga bekerja sesuai dengan konsep circular, di mana program mampu menampilkan seluruh isi linked list tanpa mengalami error. Dengan demikian, dapat disimpulkan bahwa implementasi metode tambah depan pada struktur SLLC telah berhasil menghasilkan output yang sesuai dengan teori, yaitu urutan data terbalik dari proses input.

SOAL 3

Lakukan tambah data belakang 3, 7, 1, 4, 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Pembahasan



Gambar 3 Screenshot Output Soal 3

Pada soal nomor 3 ini, proses yang dilakukan merupakan lanjutan dari soal sebelumnya, sehingga data awal dalam linked list masih berisi hasil dari nomor 2, yaitu 12, 10, 9, 7, 4, 3. Selanjutnya, pengguna melakukan operasi tambah belakang dengan memilih menu 2 secara berulang dan memasukkan data 3, 7, 1, 4, dan 3 satu per satu. Setiap kali data dimasukkan, program akan kembali ke menu utama, sehingga input memang harus dilakukan secara bertahap sesuai alur program.

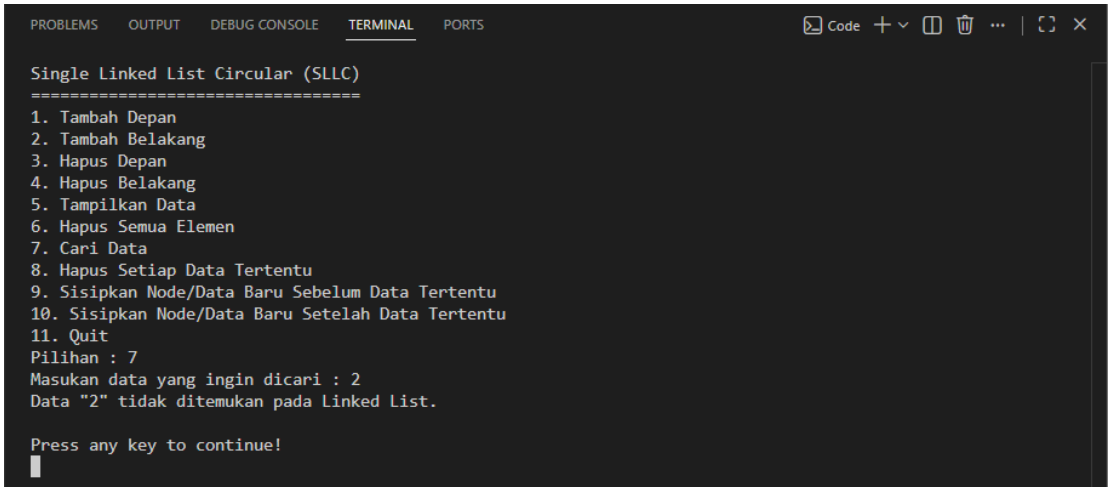
Berbeda dengan metode tambah depan, pada metode tambah belakang setiap data baru akan ditempatkan di posisi paling akhir (tail). Hal ini menyebabkan urutan data yang dimasukkan akan tetap terjaga sesuai urutan input. Secara bertahap, data baru akan ditambahkan setelah elemen terakhir tanpa mengubah posisi data sebelumnya. Dengan demikian, setelah semua proses penambahan selesai, susunan linked list akan menjadi 12, 10, 9, 7, 4, 3, 3, 7, 1, 4, 3.

Ketika fungsi tampilkan data dijalankan, program akan menelusuri seluruh node mulai dari head hingga kembali lagi ke head karena struktur bersifat circular. Hasil output yang ditampilkan akan menunjukkan seluruh data lama ditambah data baru di bagian belakang sesuai urutan input. Hal ini membuktikan bahwa fungsi tambah belakang bekerja dengan benar, yaitu menambahkan elemen tanpa mengubah urutan sebelumnya, serta menjaga hubungan antar node tetap konsisten dalam bentuk melingkar.

SOAL 4

Apa yang terjadi jika mencari angka 2 pada Single Linked List Circular (SLLC) pada data yang telah ditambahkan/dimasukkan sebelumnya dan screenshoot hasilnya

A. Pembahasan



Gambar 4 Screenshot Output Soal 4

Pada soal nomor 4 ini, proses yang dilakukan adalah mencari suatu data tertentu dalam struktur Single Linked List Circular (SLLC), yaitu angka 2, dengan memilih menu 7 (Cari Data). Pencarian ini dilakukan berdasarkan data yang telah dimasukkan sebelumnya pada soal nomor 2 dan 3, yaitu 12, 10, 9, 7, 4, 3, 3, 7, 1, 4, 3. Program kemudian akan menelusuri seluruh node dimulai dari head dan bergerak mengikuti pointer next hingga kembali lagi ke head, sesuai dengan sifat circular.

Selama proses pencarian, setiap node akan dibandingkan dengan nilai yang dicari, yaitu angka 2. Namun, karena angka 2 memang tidak pernah dimasukkan pada proses sebelumnya, maka tidak ada satu pun node yang memiliki nilai tersebut. Setelah seluruh data selesai ditelusuri dan tidak ditemukan kecocokan, program akan memberikan output berupa pesan "Data '2' tidak ditemukan pada Linked List.".

Hal ini menunjukkan bahwa algoritma pencarian bekerja dengan benar, karena mampu menelusuri seluruh elemen dalam linked list secara menyeluruh sebelum mengambil kesimpulan. Selain itu, kondisi ini juga membuktikan bahwa program memiliki penanganan yang baik ketika data yang dicari tidak tersedia, sehingga tidak terjadi error dan tetap memberikan informasi yang jelas kepada pengguna. Dengan demikian, proses pencarian dalam SLLC telah berjalan sesuai dengan konsep, yaitu melakukan penelusuran penuh (traversal) dan memberikan hasil berdasarkan keberadaan data dalam struktur tersebut.

SOAL 5

Coba cari angka 7 dan screenshoot hasilnya !

A. Pembahasan

Gambar 5 Screenshot Output Soal 5

Pada soal nomor 5 ini, proses yang dilakukan adalah mencari data tertentu dalam struktur Single Linked List Circular (SLLC), yaitu angka 7, dengan memilih menu 7 (cari data). Berbeda dengan soal sebelumnya yang mencari angka 2 dan tidak ditemukan, pada kasus ini angka 7 memang terdapat dalam data hasil dari soal nomor 2 dan 3, yaitu 12, 10, 9, 7, 4, 3, 3, 7, 1, 4, 3.

Ketika pengguna memasukkan angka 7 sebagai data yang dicari, program akan melakukan proses penelusuran (traversal) mulai dari node pertama (head) dan bergerak mengikuti pointer next hingga kembali lagi ke head. Selama proses ini, setiap node akan dibandingkan dengan nilai yang dicari. Karena terdapat lebih dari satu nilai 7 dalam linked list, maka program tidak hanya menemukan satu data saja, tetapi menandai semua data yang sesuai.

Hasil output yang ditampilkan menunjukkan bahwa setiap angka 7 diberi tanda kurung siku, seperti [7], sehingga memudahkan pengguna untuk melihat posisi data yang dicari di dalam keseluruhan list. Tampilan seperti 12 10 9 [7] 4 3 3 [7] 1 4 3 membuktikan bahwa algoritma pencarian tidak berhenti pada satu temuan saja, tetapi mampu mengidentifikasi semua kemunculan data tersebut dalam linked list.

SOAL 6

Lakukan hapus belakang dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Pembahasan

Gambar 6 Screenshot Output Soal 6

Pada soal nomor 6 ini, proses yang dilakukan adalah menghapus data pada bagian belakang (Hapus Belakang) dari struktur Single Linked List Circular (SLLC), kemudian menampilkan kembali seluruh isi data yang tersisa. Berdasarkan langkah yang dilakukan, pengguna memilih menu 4 (Hapus Belakang), sehingga program akan menghapus node terakhir yang ditunjuk oleh pointer tail. Setelah itu, pengguna memilih menu 5 (Tampilkan Data) untuk melihat hasil perubahan pada linked list.

Sebelum proses penghapusan dilakukan, data dalam linked list merupakan hasil dari soal sebelumnya, yaitu 12, 10, 9, 7, 4, 3, 3, 7, 1, 4, 3. Ketika operasi hapus belakang dijalankan, maka elemen terakhir yaitu 3 akan dihapus dari struktur. Program kemudian akan memperbarui pointer tail ke node sebelumnya, serta tetap menjaga hubungan circular dengan menghubungkan tail yang baru ke head.

Setelah proses tersebut, ketika data ditampilkan kembali, urutan linked list menjadi 12, 10, 9, 7, 4, 3, 3, 7, 1, 4. Hasil ini menunjukkan bahwa hanya satu elemen terakhir yang dihapus, sementara data lainnya tetap utuh dan tidak berubah posisinya. Hal ini membuktikan bahwa fungsi hapus belakang bekerja dengan benar, yaitu menghapus node terakhir tanpa merusak struktur linked list.

Dengan demikian, dapat disimpulkan bahwa algoritma hapus belakang pada SLLC telah berjalan sesuai konsep, di mana node terakhir dihapus dan pointer diperbarui dengan benar, serta hasil output yang ditampilkan sesuai dengan perubahan data yang terjadi

SOAL 7

Lakukan hapus setiap angka 3 dan kemudian lakukan tampilkan data lalu screenshoot hasilnya !

A. Pembahasan

Gambar 7 Screenshot Output Soal 7

Pada soal nomor 7 ini, proses yang dilakukan adalah menghapus setiap data tertentu, yaitu semua angka 3 yang terdapat dalam struktur Single Linked List Circular (SLLC). Langkah yang dilakukan adalah memilih menu 8 (Hapus Setiap Data Tertentu), kemudian memasukkan nilai 3 sebagai data yang ingin dihapus. Setelah proses penghapusan selesai, pengguna memilih menu 5 (Tampilkan Data) untuk melihat hasil perubahan pada linked list.

Sebelum penghapusan dilakukan, data dalam linked list merupakan hasil dari soal sebelumnya setelah hapus belakang, yaitu 12, 10, 9, 7, 4, 3, 3, 7, 1, 4. Ketika proses hapus setiap data tertentu dijalankan, program akan menelusuri seluruh node satu per satu. Setiap node yang memiliki nilai 3 akan dihapus dari struktur, baik yang berada di tengah maupun posisi lainnya. Dalam kasus ini, terdapat dua buah angka 3 yang berada di bagian tengah linked list, sehingga keduanya dihapus.

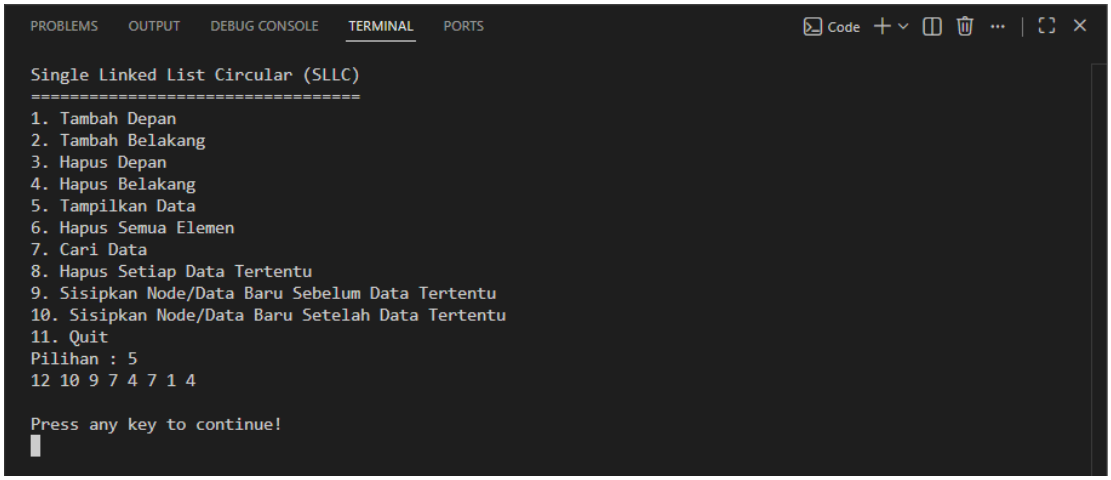
Setelah penghapusan selesai, struktur linked list akan tersusun kembali secara otomatis karena pointer antar node diperbarui. Posisi data lainnya tetap mempertahankan urutan semula, hanya saja ruang yang sebelumnya ditempati oleh angka 3 akan tertutup oleh hubungan antar node yang baru. Hasil akhirnya menjadi 12, 10, 9, 7, 4, 7, 1, 4.

Hal ini menunjukkan bahwa algoritma penghapusan data tertentu bekerja dengan baik, karena mampu menghapus lebih dari satu data dengan nilai yang sama tanpa merusak struktur circular. Selain itu, pointer antar node tetap terhubung dengan benar sehingga linked list tetap utuh dan dapat ditampilkan kembali dengan urutan yang sesuai.

SOAL 8

Tampilkan data lalu jelaskan yang mana head dan yang mana tail.

A. Pembahasan



Gambar 8 Screenshot Output Soal 8

Pada soal nomor 8 ini, dilakukan proses menampilkan data dengan memilih menu 5 (Tampilkan Data). Data yang muncul merupakan hasil akhir dari seluruh proses sebelumnya, mulai dari penambahan data depan pada soal 2, penambahan data belakang pada soal 3, pencarian data dari soal 4 dan 5, hingga penghapusan belakang pada soal 6 juga penghapusan data tertentu pada soal 7. Berdasarkan hasil yang ditampilkan pada layar, urutan data dalam Single Linked List Circular (SLLC) adalah 12 10 9 7 4 7 1 4.

Dalam struktur SLLC, head adalah node pertama dalam linked list, yaitu data yang pertama kali ditunjuk saat traversal dimulai. Pada hasil tersebut, head berada pada angka 12, karena traversal dimulai dari data tersebut. Sementara itu, tail adalah node terakhir sebelum kembali lagi ke head. Pada data tersebut, tail berada pada angka 4 yang paling kanan.

Karena sifatnya circular, maka node tail tidak bernilai NULL, melainkan menunjuk kembali ke head. Artinya, setelah angka 4 (tail), pointer akan kembali ke 12 (head), sehingga membentuk siklus. Hal ini yang membedakan SLLC dengan linked list biasa, dapat disimpulkan bahwa Head adalah 12 data pertama dan Tail adalah 4 data terakhir sebelum kembali ke head

Struktur ini memastikan bahwa semua data saling terhubung dalam bentuk lingkaran, sehingga traversal bisa dilakukan terus menerus tanpa batas selama tidak dihentikan oleh kondisi tertentu dalam program.

SOAL 9

Jika baris ke 103 dan 104 dihapus maka apa yang akan terjadi pada saat memasukkan data, dan jelaskan mengapa?

A. Pembahasan

maka fungsi isEmpty() tidak lagi mengembalikan nilai yang jelas (undefined behavior). Hal ini akan berdampak langsung pada proses saat memasukkan data ke dalam linked list.

Secara konsep, fungsi isEmpty() digunakan sebagai penentu kondisi awal apakah linked list kosong atau tidak. Fungsi ini dipanggil pada berbagai bagian penting seperti tambahDepan() dan tambahBelakang(). Ketika program pertama kali dijalankan, kondisi awal head adalah NULL, sehingga fungsi isEmpty() seharusnya mengembalikan nilai 1 (benar/kosong). Nilai ini kemudian digunakan untuk menentukan bahwa node pertama yang dimasukkan harus menjadi head sekaligus tail.

Namun, jika baris 103 dan 104 dihapus, maka fungsi isEmpty() tidak memiliki perintah return. Akibatnya, nilai yang dikembalikan oleh fungsi tersebut menjadi tidak terdefinisi (acak) tergantung kondisi memori saat itu. Dampaknya, ketika pengguna memasukkan data pertama, program bisa saja menganggap bahwa linked list tidak kosong, padahal sebenarnya kosong.

Jika kondisi ini terjadi, maka program akan langsung masuk ke bagian else pada fungsi tambahDepan() atau tambahBelakang(). Di bagian ini terdapat operasi seperti, baru->next = head; dan tail->next = head;

Padahal head dan tail masih bernilai NULL. Hal ini menyebabkan program mencoba mengakses pointer yang belum valid, sehingga berpotensi menimbulkan error serius seperti segmentation fault atau crash.

Selain itu, hubungan antar node juga tidak akan terbentuk dengan benar karena node pertama tidak pernah diinisialisasi sebagai head dan tail. Akibatnya, seluruh operasi selanjutnya seperti menampilkan data, menghapus data, atau menyisipkan data akan menjadi tidak stabil atau bahkan gagal dijalankan.

Kesimpulannya, penghapusan baris 103 dan 104 menyebabkan fungsi isEmpty() kehilangan fungsinya sebagai pengontrol kondisi awal. Hal ini membuat logika program menjadi tidak terarah dan berpotensi menyebabkan kesalahan fatal saat proses input data pertama, karena program tidak dapat membedakan antara kondisi list kosong dan tidak kosong.

SOAL 10

Jelaskan apa itu variabel head dan tail pada sllc!

A. Pembahasan

Pada struktur data Single Linked List Circular (SLLC), variabel head dan tail memiliki peran yang sangat penting sebagai penunjuk utama dalam pengelolaan data.

Variabel head adalah pointer yang digunakan untuk menunjuk ke node pertama dalam linked list. Artinya, setiap kali kita ingin mengakses atau menelusuri data dalam list, proses tersebut selalu dimulai dari head. Head menjadi titik awal dari seluruh rangkaian node yang saling terhubung. Jika linked list dalam kondisi kosong, maka nilai head adalah NULL, yang menandakan belum ada data yang tersimpan.

Sementara itu, variabel tail adalah pointer yang menunjuk ke node terakhir dalam linked list. Tail digunakan untuk mempermudah proses penambahan data di bagian akhir (belakang), karena kita tidak perlu menelusuri seluruh node dari awal. Dengan adanya tail, proses penambahan di belakang menjadi lebih efisien.

Hubungan antara head dan tail pada SLLC memiliki ciri khas yang membedakannya dari linked list biasa, yaitu sifat circular (melingkar). Pada SLLC, node terakhir yang ditunjuk oleh tail tidak memiliki nilai NULL pada bagian next, melainkan akan menunjuk kembali ke node pertama, yaitu head. Dengan kata lain, terdapat hubungan penting yaitu tail->next = head

Hubungan ini membuat struktur data menjadi berbentuk lingkaran, sehingga proses penelusuran data dapat dilakukan terus-menerus tanpa batas akhir sampai kembali ke titik awal.

Secara keseluruhan, head berfungsi sebagai pintu masuk untuk membaca data, sedangkan tail berfungsi sebagai penanda akhir sekaligus penghubung kembali ke awal. Keduanya harus selalu dijaga konsistensinya, karena jika salah satu tidak diperbarui dengan benar saat terjadi operasi seperti tambah atau hapus, maka struktur linked list bisa rusak atau tidak dapat diakses dengan benar.

LINK GITHUB

https://github.com/Arthur-369/PRAKTIKUM-Algoritma-dan-Struktur-Data